

ECOM EXPERTS

Compliance Pilot Hub

Manual de usuario e instalación

eucompliance-hub 1.0.0 · 06/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Panel de reparación (sin sesión)

5. Cómo se usa, día a día

Administrador

Comerciante (cliente del Hub)

Módulo PrestaShop ecom_eucompliance

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Seguridad

Qué se ha revisado

Qué se ha arreglado en esta revisión

Qué queda pendiente

9. Ficha técnica

Este proyecto no es un módulo de PrestaShop. Es una aplicación web completa (Symfony 7) que se instala en un servidor propio. No se ha instalado en las tiendas locales ni hay capturas de pantalla: el paquete no incluye la carpeta `vendor/` (se genera con `composer install` en el servidor) y necesita MariaDB, Redis y, opcionalmente, un servicio Node.js con Chromium, así que no se puede levantar en local sin montar toda esa infraestructura.

1. Qué hace y para quién

Compliance Pilot Hub es el servidor central de un servicio de **alertas de cumplimiento normativo europeo para comercios electrónicos**. Recoge normativas de fuentes oficiales (boletines RSS del BOE, EUR-Lex, AEPD, etc.), las clasifica y resume con inteligencia artificial (Gemini, Claude u OpenAI, por orden de prioridad configurable), y las difunde en un portal público con fichas explicativas, glosario, comparativas, blog, herramientas (calculadora de multas RGPD, IVA OSS por país) y un widget insertable en otras webs.

Para el comerciante registrado ofrece un **área de cuenta**: perfil de negocio (sector, países donde vende, tipo de productos), normativas que le afectan según ese perfil, generación de textos legales orientativos, una **auditoría web** de su tienda (cookies, cabeceras de seguridad, páginas legales, rastreadores) con informe en PDF, y una clave de API para conectar el módulo de PrestaShop `ecom_eucompliance`.

Para el administrador hay un **panel bajo una URL secreta** (`/panel-xxxx`) desde el que se gestionan fuentes RSS, normativas (publicar, descartar, enriquecer con IA, bloquear), el sistema de Pasaporte Digital de Producto (DPP), usuarios, tiendas conectadas, claves de API, blog, preguntas de la comunidad, ajustes (SMTP, IA, pagos, SEO, caché) y las tareas de mantenimiento.

Todo el sistema es **orientativo y no vinculante**: cada usuario y cada módulo conectado deben aceptar el descargo legal, cuya aceptación queda registrada (fecha, IP, versión y hash del texto) y notificada por correo.

2. Compatibilidad

Componente	Requisito
PHP	8.2 o superior, con <code>ctype</code> , <code>iconv</code> , <code>curl</code> , <code>pdo_mysql</code> , <code>intl</code> , <code>mbstring</code> , <code>openssl</code> (y <code>redis</code> si se usa Redis)
Framework	Symfony 7.0 (se instala con <code>composer install --no-dev</code>)
Base de datos	MariaDB 10.11 o superior (Doctrine ORM + migraciones)
Colas y caché	Redis 7 (transporte de Symfony Messenger; la caché puede ir en Redis o en ficheros)
Auditoría web con navegador real	Node.js 20 + Playwright + Chromium (servicio <code>headless/server.js</code> , opcional)
Servidor web	nginx o Apache. Incluye plantillas para HestiaCP y un <code>.htaccess</code> para hosting compartido (raíz del proyecto como docroot)
Correo	Cualquier SMTP (se configura desde el panel)
Pagos	PayPal Subscriptions (webhook firmado). Redsys tiene los ajustes previstos pero sin flujo implementado
Idioma	Interfaz en castellano
Usuarios	Multiusuario con roles: <code>ROLE_SUPER_ADMIN</code> , <code>ROLE_STAFF</code> , <code>ROLE_AGENCY</code> , <code>ROLE_MERCHANT</code> , <code>ROLE_LEAD</code>

No tiene concepto de multitienda de PrestaShop: cada tienda conectada es un registro `sites` con su `site_id` y `site_secret`.

3. Instalación

La guía detallada está en `deploy/INSTALL-HESTIA.md` (VPS con HestiaCP) y `deploy/INSTALL-HEADLESS-HESTIA.md` (servicio de navegador). Resumen:

- Cuenta y dominio en Hestia:** usuario, dominio web con SSL (Let's Encrypt, forzar HTTPS), base de datos.
- Dependencias que Hestia no trae:** extensiones PHP (`redis`, `intl`, `bcmath`, `gd`, `zip`), `redis-server`, Composer y, si se quiere auditoría con navegador, Node.js 20 + Chromium.
- Código:** subir el proyecto a `~/web/<dominio>/repo`, enlazar `public_html` a `repo/public`, ejecutar `composer install --no-dev --optimize-autoloader`, crear `var/cache` `var/log` `var/uploads` `var/install` con permisos 770.
- .env.local** con `APP_ENV=prod`, `APP_SECRET`, `DATABASE_URL`, `REDIS_URL` y `EUCOMP_USE_CACHE=0` (permisos 600). El `.env` del paquete solo trae valores por defecto sin secretos.
- Base de datos:** `php bin/console doctrine:migrations:migrate --no-interaction` y `php bin/console cache:warmup --env=prod`. Además, cualquier pantalla del panel detecta columnas que falten y ofrece un

- botón «Aplicar migraciones» (`SchemaMismatchListener`), y el panel de reparación tiene la misma acción.
6. **Plantilla nginx** `deploy/templates/hestia-eucomply.tpl/.stpl` copiada a las plantillas de Hestia y asignada al dominio (imprescindible: las genéricas no entienden el controlador frontal de Symfony).
 7. **Servicios systemd** desde `deploy/templates/`: `eucomply-messenger@async`, `eucomply-messenger@rss` (workers de colas), `eucomply-cron.timer` (cada 5 minutos ejecuta `regulations:cron`) y, opcional, `eucomply-headless`.
 8. **Asistente web** en `https://<dominio>/install`, siete pasos: comprobaciones, dominio, base de datos, servicios (SMTP, IA, PayPal, Redsys, Meilisearch, headless), administrador (usuario, contraseña y slug del panel generados al azar, aceptación del descargo), 2FA y finalización. Al terminar escribe `INSTALLED=true` y `/install` deja de responder. Las credenciales quedan en `var/install/INSTALL_RECOVERY.md`: **copiarlas y borrar el fichero**.
 9. **Cron nocturno** (DPP y valores por campo con IA): llamar una vez al día a `GET /api/cron/nightly?token=...` desde el cron de Hestia o un servicio externo. El token se genera en el panel, apartado «Cron nocturno».

Tablas que crea (`migrations/` y `src/Service/SchemaProvider.php`): `users`, `sites`, `regulations`, `regulation_blocklist`, `regulation_field_values`, `regulation_audit_result`, `audit_report`, `audit_log`, `rss_sources`, `settings`, `blog_posts`, `community_questions`, `glossary`, `newsletter_subscribers`, `messenger_messages`.

Desinstalación: no hay desinstalador. Parar y deshabilitar los servicios systemd, borrar la base de datos, el usuario de Hestia (o la carpeta del proyecto) y la entrada del cron externo.

4. Configuración

No hay capturas (ver nota inicial). Todo se configura desde `https://<dominio>/panel-<slug>/settings`, con una pestaña por sección y un botón «Guardar» por sección. Los valores se guardan en la tabla `settings` y tienen prioridad sobre las variables de `.env.local`.

Sección	Campos	Qué hace de verdad
Sitio	Nombre del sitio, tagline, email de contacto, URL de soporte, zona horaria	Textos de cabecera y pie del portal y del correo.
SMTP	Host, puerto, cifrado (TLS / SSL / ninguno), usuario, contraseña, remitente, nombre del remitente, email para notificaciones internas	Construye el DSN del correo. Sin SMTP no salen ni las verificaciones de registro ni los avisos de aceptación del descargo.
IA	Prioridad (CSV, por ejemplo <code>gemini,openai,claude</code>), clave y modelo de Gemini, Claude y OpenAI	Orden en que se intentan los proveedores. Modelo por defecto de Gemini: <code>gemini-flash-latest</code> . Se usa para clasificar RSS, resumir normativas, explicaciones profundas, textos legales, DPP y descubrimiento de normativas.
Facturación	Moneda, PayPal (Client ID, Secret, modo), Redsys (código de comercio, terminal, clave, entorno)	PayPal se usa para crear planes y verificar el webhook. El ID del webhook de PayPal es obligatorio: sin él el Hub rechaza todos los avisos de PayPal (ver seguridad).
Planes	Nombre y precio de cada uno de los cuatro planes	Solo textos de la página <code>/precios</code> .
RSS	Palabras clave globales, presupuesto de tiempo del cron (segundos)	Filtro adicional al ingerir y tope de duración de cada pasada del cron.
SEO	Clave IndexNow, ID de Google Analytics 4, tokens de verificación de Search Console y Bing, usuario de Twitter, meta descripción global, logo para schema.org	Etiquetas del portal público; con IndexNow se avisa a los buscadores al publicar.
Caché	Caché activa, driver, host, puerto, base, contraseña, prefijo, TTL, caché de página completa (FPC) y su TTL	Caché de la aplicación y cabeceras <code>Cache-Control</code> públicas que emite <code>PublicCacheHeadersListener</code> .
Módulo	Nombre, URL de venta, descripción, texto del botón, logo	Bloque promocional del módulo de PrestaShop en la ficha de cada normativa.

Otras pantallas de configuración del panel:

- **Sistema > Mantenimiento > Entorno:** editor de `.env.local` (debug, caché de Symfony, regenerar slug del panel, reparar permisos, reparar esquema de BD).
- **Cron nocturno:** genera o regenera el token de `/api/cron/nightly` y muestra la URL exacta para el cron externo.
- **Claves de API:** genera o revoca la clave de cada usuario y asigna tiendas a usuarios.

Panel de reparación (sin sesión)

`https://<dominio>/repair/panel-<slug>` funciona aunque el panel normal esté caído: muestra estado, limpia caché, aplica migraciones, activa o desactiva el modo debug y la caché de Symfony, parchea un fallo

conocido de `phpdoc-parser`, y **puede promocionar usuarios a administrador, resetear su 2FA y cambiar contraseñas**. Solo lo protege el slug. Recomendación: cuando no se esté usando, añadir `REPAIR_PANEL_ENABLED=0` a `.env.local` (añadido en esta revisión); con eso responde 404. Cada acceso queda anotado en `var/log/repair.log`.

Existe también un modo de diagnóstico sin Symfony: `https://<dominio>/?_diag=panel-<slug>` (solo si el docroot es `public/`).

5. Cómo se usa, día a día

Administrador

1. **Fuentes RSS** (`/panel-.../rss`): alta manual, «Sembrar» el catálogo de fuentes incluido (`rss:seed`), probar cada fuente, ejecutarlas una a una o todas. Hay tres buscadores asistidos: por palabra clave, en fuentes oficiales y con IA.
2. **Cron de normativas** (`/panel-.../system/cron` o `regulations:cron` cada 5 minutos por systemd): ingiere las fuentes, clasifica cada entrada con IA (familia, severidad, sector, países, resumen) y deja las normativas en estado pendiente. Botones adicionales: cargar histórico, reclasificar, descubrimiento con IA por temas, enriquecimiento con IA de fichas incompletas.
3. **Normativas** (`/panel-.../regulations`): revisar las pendientes, editar, **publicar**, descartar o bloquear (la blocklist evita que vuelvan a entrar). «Enriquecer» genera la explicación profunda que se muestra en la ficha pública. Mantenimiento: deduplicar, purgar antiguas, reparar slugs, restaurar borradas desde el registro de auditoría.
4. **DPP** (Pasaporte Digital de Producto): «Preparar» detecta las normativas que aplican, «Generar» crea con IA la estructura de datos DPP de cada una y «Valores IA» rellena los valores por campo. El cron nocturno lo hace solo, de 10 en 10.
5. **Auditorías** (`/panel-.../audit`): lanzar una auditoría de cualquier web pública, ver el historial, el JSON, el CSV, el PDF y enviarla por correo. Los volcados de depuración están en `var/log/audit/`.
6. **Usuarios y tiendas**: alta, edición, borrado, reseteo de 2FA; asignación de tiendas conectadas a su propietario.
7. **Blog y comunidad**: entradas del blog (Markdown) y moderación de las preguntas que dejan los visitantes en las fichas de normativas.
8. **Sistema**: información del servidor, estado de los workers, caché (limpiar pools, Symfony, OPcache, probar Redis), registro de auditoría de acciones (`audit_log`).

Comerciante (cliente del Hub)

1. **Registro** en `/register` con aceptación obligatoria del descargo, verificación por correo (`/verificar-email/{token}`).
2. **Primer acceso**: activación de 2FA TOTP (`/cuenta/2fa`) y asistente de bienvenida de cuatro pasos (perfil de negocio).
3. **Panel de cuenta** (`/cuenta`): normativas relevantes según su perfil, con explicación bajo demanda; textos legales orientativos generados con IA; auditoría web de su tienda con PDF; clave de API para el módulo de PrestaShop; reclamar una tienda conectada.

Módulo PrestaShop `ecom_eucompliance`

Se conecta con la clave de API del usuario (`POST /api/v1/sites/register` , autenticación Bearer) y recibe `site_id` y `site_secret` . Desde ahí firma cada petición con HMAC SHA-256 sobre `timestamp|nonce|cuerpo` (`GET /api/v1/regulations` , auditorías `/api/v1/audit` , DPP `/api/v1/dpp/*`). Contrato completo en `docs/API.md` y `docs/DOC_HUB_INTEGRATION.md` ; documentación pública en `/api/docs` .

6. Qué ve el cliente en la tienda

El Hub no pinta nada en la tienda del comerciante (eso lo hace el módulo). Lo que ve cualquier visitante del Hub es el portal público:

- Portada `/` con las últimas normativas y el aviso de no vinculación; `/precios` .
- Catálogo `/normativas` , por familia `/normativas/familia/{familia}` , ficha `/normativas/{id}-{slug}` con explicación profunda, preguntas de la comunidad y bloque del módulo.
- Landings SEO `/cumplimiento/{tema}` y `/cumplimiento/{producto}/{país}` , comparativas `/comparar` , glosario `/glosario` , blog `/blog` , herramientas `/herramientas` .
- Newsletter con doble confirmación y guías en PDF por familia (`/recursos/guia-{familia}.pdf`).
- Widget insertable: `<script src="https://<dominio>/widget.js" data-family="datos" data-limit="5"></script>` , que consume la API pública `/api/v1/regulations` .
- Páginas legales (`/legal/*`), sitemaps, `robots.txt` , manifiesto PWA e imágenes Open Graph generadas.

7. Preguntas frecuentes y problemas

La web da error 500 tras subir ficheros nuevos. Es la caché compilada de Symfony. Con `EUCOMP_USE_CACHE=0` (por defecto) se purga en cada petición; si está activada, entrar en `/repair/panel-<slug>` y pulsar «Limpiar caché» o «Fix PhpDocParser» (que además hace el warmup).

He perdido la URL del panel. Está en `ADMIN_PANEL_SLUG` de `.env.local` . Para cambiarla: `sudo bash ops/regenerate-admin-slug.sh` (ahora genera 32 caracteres hexadecimales).

El administrador ha perdido el 2FA. `sudo bash ops/reset-2fa-admin.sh <usuario>` o, desde el panel de reparación, «Reset 2FA».

Aparece «Petición rechazada, la petición no procede de este sitio». Un formulario se ha enviado desde otro dominio (protección anti-CSRF añadida en esta revisión). Volver a la página y repetir. Si ocurre en uso normal, comprobar que `APP_URL` y el dominio real coinciden y que el proxy pasa la cabecera `Host` correcta (`TRUSTED_PROXIES`).

La auditoría web dice «No se auditan direcciones internas». Solo se auditan dominios públicos; `localhost` , IPs privadas o reservadas se rechazan a propósito.

«Headless service no disponible». El servicio `eucomply-headless` no está arrancado o `HEADLESS_SECRET` no coincide entre `.env.local` y el servicio. `systemctl status eucomply-headless` .

PayPal responde 503 `webhook_not_configured` . Falta el ID del webhook en Ajustes > Facturación. Sin él no se puede verificar la firma y el Hub no procesa nada.

El módulo recibe 401 `replay_nonce` . Ha repetido un `X-Eucomp-Nonce` dentro de diez minutos. Cada petición debe llevar un nonce nuevo, como indica `docs/DOC_HUB_INTEGRATION.md` .

Una pantalla del panel muestra «El esquema de BD está desactualizado». Falta una columna nueva; pulsar «Aplicar migraciones ahora» (solo añade columnas y tablas, no borra datos).

8. Seguridad

Qué se ha revisado

El controlador frontal (`index.php` y `public/index.php`), la configuración de seguridad de Symfony (`config/packages/security.yaml` , `framework.yaml`), los 49 controladores (`src/Controller/**`), el autenticador HMAC, los servicios de auditoría web y de IA, las 106 plantillas Twig (uso de `|raw` , variables dentro de JavaScript), el servicio Node del navegador (`headless/server.js`), los `.htaccess` , los scripts de `ops/` y `deploy/` y los ficheros `.env` .

Resultado positivo: consultas a base de datos a través de Doctrine con parámetros (las sentencias sueltas del mantenimiento salen de listas fijas del código); Markdown de la IA convertido con escape previo de HTML; contraseñas con bcrypt; 2FA TOTP; API con HMAC y ventana de cinco minutos; webhook de PayPal verificado con la API de PayPal; servicio headless solo en `127.0.0.1` con secreto; sin secretos en el código ni en el `.env` del paquete; sin `eval` ni `unserialize` ; carpetas sensibles bloqueadas por `.htaccess` .

Qué se ha arreglado en esta revisión

Fichero	Problema	Arreglo
<code>src/EventListener/OriginCheckListener.php</code> (nuevo)	Ningún formulario del panel ni de la cuenta lleva token CSRF (0 llamadas a <code>isCsrfTokenValid</code> en unas 100 rutas POST); solo protegía la cookie <code>SameSite=Lax</code> .	Comprobación global de origen: toda petición POST/PUT/PATCH/DELETE fuera de <code>/api/</code> cuya cabecera <code>Origin</code> o <code>Referer</code> apunte a otro dominio se rechaza con 403.
<code>src/Security/ApiHmacAuthenticator.php</code>	El nonce se firmaba pero no se comprobaba: una petición capturada se podía repetir durante cinco minutos.	Nonce guardado en la caché de la aplicación diez minutos; si se repite, 401 <code>replay_nonce</code> .
<code>src/Controller/Install/InstallController.php</code> , <code>ops/regenerate-admin-slug.sh</code>	El slug del panel tenía 8 caracteres hexadecimales (32 bits): fuerza bruta viable contra <code>/repair/panel-...</code> , que no exige sesión.	32 caracteres (128 bits) en instalaciones y regeneraciones nuevas. Las instalaciones existentes deben regenerarlo con el script.
<code>src/Controller/RepairController.php</code>	Panel de reparación sin sesión con capacidad de dar permisos de administrador y cambiar contraseñas; sin rastro de accesos; comparación del token no constante.	<code>hash_equals</code> , interruptor <code>REPAIR_PANEL_ENABLED=0</code> en <code>.env.local</code> (responde 404) y registro de cada acceso en <code>var/log/repair.log</code> .
<code>src/Controller/Public/PayPalWebhookController.php</code>	Si no había ID de webhook configurado, se aceptaban avisos sin verificar la firma.	Sin ID configurado responde 503 y no procesa nada; con firma inválida, 401.
<code>src/Service/WebAuditor.php</code>	La auditoría web aceptaba cualquier URL, incluidas <code>localhost</code> e IPs privadas: un usuario registrado podía	Solo <code>http(s)</code> sin credenciales, y el nombre se resuelve y se rechaza si apunta a rangos privados, de bucle local o reservados.

Fichero	Problema	Arreglo
	hacer que el servidor consultara servicios internos (SSRF).	
<code>templates/public/base.html.twig</code>	El ID de Google Analytics se insertaba dentro de JavaScript sin escape de JS.	<code> e('js')</code> .
<code>.htaccess</code> (raíz)	No bloqueaba <code>.git/</code> si el proyecto se despliega clonado en hosting compartido.	Regla <code>^\.git</code> denegada.
<code>deploy/templates/eucomply-cron.service</code>	Ejecutaba <code>app:cron:run</code> , un comando que no existe: el temporizador fallaba cada cinco minutos.	Ejecuta <code>regulations:cron</code> , que es el cron real.

Todos los ficheros PHP tocados pasan `php -l` con PHP 8.2. No se ha podido ejecutar la aplicación (sin `vendor/` ni base de datos), así que estos cambios deben probarse en el servidor: entrar en el panel, guardar un ajuste, lanzar una auditoría y hacer un handshake desde el módulo.

Qué queda pendiente

- **Tokens CSRF por formulario.** La comprobación de origen es una segunda barrera; lo correcto es añadir `csrf_token()` a cada formulario y validarlo en cada acción. Son unas 100 rutas y 60 plantillas: cambio grande, no abordado.
- **Límite de intentos de login.** `docs/SECURITY.md` dice «5 intentos / 15 min / IP», pero no está implementado (no hay `login_throttling` ni `symfony/rate-limiter` en `composer.json`). Añadir el paquete y `login_throttling` al firewall `main`.
- **Panel de reparación.** Sigue sin exigir sesión por diseño. Conviene restringirlo además por IP en nginx (`location ^~ /repair/ { allow <ip>; deny all; }`) y dejar `REPAIR_PANEL_ENABLED=0` fuera de las emergencias.
- **Documentación desfasada.** `README.md` y `docs/INSTALL.md` citan scripts `deploy/01-...10-*.sh` que no están en el paquete (solo la guía de HestiaCP); `docs/BILLING.md` habla de `/webhooks/paypal` y `app:billing:retry`, que no existen (la ruta real es `/api/paypal/webhook`); Redsys no tiene flujo de cobro implementado.
- **Cabeceras de seguridad en hosting compartido.** El `.htaccess` de la raíz no añade `X-Content-Type-Options` ni `X-Frame-Options` (se evitó `Header` por compatibilidad con Plesk). En Hestia las pone la plantilla nginx; en otros hostings, añadirlas en el servidor.

- **Registro público sin captcha ni límite.** `/register`, `/newsletter/suscribirse` y `/normativas/{id}/preguntar` aceptan envíos sin protección anti-bots; la moderación manual mitiga el último.
- **Textos sin tildes.** Buena parte de las plantillas y mensajes están escritos sin acentos ni eñes (faltan las tildes de palabras como configuración o auditoría). No se ha corregido por el volumen (106 plantillas) y porque no era el objetivo de esta revisión.

9. Ficha técnica

Estructura: `src/` (Controller, Entity, Repository, Service, Security, EventListener, Command, Message/MessageHandler, Twig), `templates/`, `config/`, `migrations/`, `public/` (docroot recomendado), `index.php` + `.htaccess` (docroot alternativo para hosting compartido), `headless/` (servicio Node), `deploy/` (guías y plantillas de Hestia y systemd), `ops/` (backup, health-check, update, regenerar slug, reset 2FA), `docs/`.

Tablas: `users`, `sites`, `regulations`, `regulation_blocklist`, `regulation_field_values`, `regulation_audit_result`, `audit_report`, `audit_log`, `rss_sources`, `settings`, `blog_posts`, `community_questions`, `glossary`, `newsletter_subscribers`, `messenger_messages`.

Firewalls de Symfony: `install` (sin seguridad, solo hasta `INSTALLED=true`), `api` (`/api/v1`, sin estado, `ApiHmacAuthenticator`), `main` (formulario de login con CSRF, 2FA). Control de acceso: `/cuenta` y `/dashboard` requieren `ROLE_USER`; `/panel-...` requiere `ROLE_STAFF`; `/repair/` es público.

Grupos de rutas: público (`/`, `/normativas`, `/blog`, `/glosario`, `/comparar`, `/cumplimiento`, `/herramientas`, `/legal`, `/precios`, `/widget.js`, sitemaps), cuenta (`/cuenta/*`), registro y login (`/register`, `/login`, `/verificar-email`), panel (`/panel-<slug>/*`), API (`/api/v1/*`, `/api/docs`, `/api/search/instant`, `/api/cron/nightly`, `/api/paypal/webhook`), reparación (`/repair/panel-<slug>/*`), instalación (`/install/*`).

Comandos de consola: `regulations:cron` (ingesta RSS con cursor y presupuesto de tiempo, normalización nocturna), `rss:seed`, `rss:ingest`, `rss:classify`, `regulations:reclassify`, `regulations:backfill-slug`, `app:regulations:regenerate-all`, `app:dpp:generate-missing`, `app:dpp:generate-values`, `app:audit:scheduled`.

Colas (Messenger): transportes `async`, `rss` y `headless` sobre Redis; `failed` sobre Doctrine. Mensajes: `IngestRssMessage`, `HeadlessScanMessage`, `ClassifyRegulationMessage`, `NotifyDisclaimerMessage`.

Servicios systemd: `eucomply-messenger@async`, `eucomply-messenger@rss`, `eucomply-messenger@headless` (opcional), `eucomply-cron.timer` (cada 5 min, `regulations:cron`), `eucomply-headless` (Node, `127.0.0.1:9223`, cabecera `X-Headless-Secret`).

Cron externo: `GET /api/cron/nightly?token=<token>&dpp_limit=10&values_limit=10`, una vez al día; tope de 25 minutos por ejecución.

Variables de `.env.local`: `APP_ENV`, `APP_DEBUG`, `APP_SECRET`, `INSTALLED`, `APP_URL`, `ADMIN_PANEL_SLUG`, `REPAIR_PANEL_ENABLED` (nuevo, `1` por defecto), `DATABASE_URL`, `REDIS_URL`, `MESSENGER_TRANSPORT_DSN`, `MAILER_DSN`, `MAIL_FROM`, `MAIL_FROM_NAME`, `DISCLAIMER_NOTIFY_EMAIL`, `GEMINI_API_KEY`, `CLAUDE_API_KEY`, `OPENAI_API_KEY` y sus modelos, `HEADLESS_URL`, `HEADLESS_SECRET`, `PAYPAL_*`, `REDSYS_*`, `TOTP_ISSUER`,

`MEILISEARCH_*`, `UPLOADS_PATH`, `TRUSTED_PROXIES`, `TRUSTED_HOSTS`, `EUCOMP_USE_CACHE`. Los ajustes del panel (tabla `settings`) tienen prioridad sobre estas variables.

Registros: `var/log/prod.log` (Symfony), `var/log/audit/*.json` (volcado de cada auditoría web), `var/log/repair.log` (accesos al panel de reparación), tabla `audit_log` (acciones de usuarios y eventos de sistema), `journalctl -u 'eucomply-*` (workers y cron).

Permisos: `var/` con escritura para el usuario de PHP-FPM; `.env.local` a 600; los `var/cache` se recrean a 0777 desde el panel de reparación cuando se limpia la caché.